AI Assisted Coding

Lab Assignment 12.3

Name : V Sai Krishnachaithanya **Date : 13/03/2026**

Hall Ticket no : 2303A52154

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
o Quick Sort
o Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the top 10 students based on CGPA.

Expected Outcome

• Correctly sorted student records.
• Performance comparison between Quick Sort and Merge Sort.
• Clear output of top-performing students.

Prompt :

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.analyse the above senario and generate a program that stores student records (Name, Roll Number, CGPA).implement using merge sort and quick sort and print a function to display the top 10 students based on CGPA.

```python
# SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student record
# analyse the above senario and generate a program that stores student records (Name, Roll Number, CGPA).implement using merge s
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

    def __repr__(self):
        return f"{self.name} (Roll No: {self.roll_number}, CGPA: {self.cgpa})"
def merge_sort(students):
    if len(students) <= 1:
        return students

    mid = len(students) // 2
    left_half = merge_sort(students[:mid])
    right_half = merge_sort(students[mid:])

    return merge(left_half, right_half)
def merge(left, right):
    sorted_students = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i].cgpa > right[j].cgpa:
            sorted_students.append(left[i])
            i += 1
        else:
            sorted_students.append(right[j])
            j += 1

    sorted_students.extend(left[i:])
    sorted_students.extend(right[j:])

    return sorted_students
def quick_sort(students):
    if len(students) <= 1:
        return students
```

Output :

```
✓ 0.0s
Students sorted by CGPA using Merge Sort:
Top 10 Students based on CGPA:
Eve (Roll No: 005, CGPA: 9.5)
Karl (Roll No: 011, CGPA: 9.4)
Ivan (Roll No: 009, CGPA: 9.3)
Alice (Roll No: 001, CGPA: 9.2)
Grace (Roll No: 007, CGPA: 9.1)
Charlie (Roll No: 003, CGPA: 9.0)
Heidi (Roll No: 008, CGPA: 8.9)
David (Roll No: 004, CGPA: 8.8)
Frank (Roll No: 006, CGPA: 8.7)
Judy (Roll No: 010, CGPA: 8.6)


Students sorted by CGPA using Quick Sort:
Top 10 Students based on CGPA:
Eve (Roll No: 005, CGPA: 9.5)
Karl (Roll No: 011, CGPA: 9.4)
Ivan (Roll No: 009, CGPA: 9.3)
Alice (Roll No: 001, CGPA: 9.2)
Grace (Roll No: 007, CGPA: 9.1)
Charlie (Roll No: 003, CGPA: 9.0)
Heidi (Roll No: 008, CGPA: 8.9)
David (Roll No: 004, CGPA: 8.8)
Frank (Roll No: 006, CGPA: 8.7)
Judy (Roll No: 010, CGPA: 8.6)
```

Task 2: Implementing Bubble Sort with AI Comments

• Task: Write a Python implementation of Bubble Sort.

• Instructions:

• Students implement Bubble Sort normally.

• Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).

• Request AI to provide time complexity analysis.

• Expected Output:

• A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

Prompt :

Write a Python implementation of Bubble Sort and generate inline comments explaining key logic and also provide time complexity of the algorithm.

```python
#Write a Python implementation of Bubble Sort and generate inline comments explaining key logic and also provide time complexity of the algorithm.
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the array
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Compare adjacent elements and swap if they are in the wrong order
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]  # Swap
                swapped = True  # Set flag to True if a swap occurred
        # If no swapping occurred, the array is already sorted
        if not swapped:
            break
    return arr
# Example usage
arr = [64, 34, 25, 12, 22, 11, 90]
sorted_arr = bubble_sort(arr)
print("Sorted array is:", sorted_arr)
print("Time Complexity of Bubble Sort: O(n^2)")
# Time Complexity: O(n^2) in the worst and average cases, O(n) in the best case (when the array is already sorted).
```

Output :

```
✓ 0.0s

Sorted array is: [11, 12, 22, 25, 34, 64, 90]
Time Complexity of Bubble Sort: O(n^2)
```

Task 3: Quick Sort and Merge Sort Comparison

• Task: Implement Quick Sort and Merge Sort using recursion.

• Instructions:

• Provide AI with partially completed functions for recursion.

• Ask AI to complete the missing logic and add docstrings.

• Compare both algorithms on random, sorted, and reverse-sorted lists.

• Expected Output:

• Working Quick Sort and Merge Sort implementations.

• AI-generated explanation of average, best, and worst-case complexities.

Prompt :

Implement Quick Sort and Merge Sort using recursion and Compare both algorithms on random, sorted, and reverse-sorted lists and print the AI-generated explanation of average, best, and worst-case complexities

```python
# Implement Quick Sort and Merge Sort using recursion and Compare both algorithms on random, sorted, and reverse-sorted lists and print the AI-generated
import random
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])
    return merge(left_half, right_half)
def merge(left, right):
    sorted_arr = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            sorted_arr.append(left[i])
            i += 1
        else:
            sorted_arr.append(right[j])
            j += 1
    sorted_arr.extend(left[i:])
    sorted_arr.extend(right[j:])
    return sorted_arr
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
```

Output :

```
✓ 0.0s

Random List: [79, 70, 42, 83, 18, 84, 46, 22, 66, 82]
Merge Sort Result: [18, 22, 42, 46, 66, 70, 79, 82, 83, 84]
Quick Sort Result: [18, 22, 42, 46, 66, 70, 79, 82, 83, 84]

Sorted List: [18, 22, 42, 46, 66, 70, 79, 82, 83, 84]
Merge Sort Result: [18, 22, 42, 46, 66, 70, 79, 82, 83, 84]
Quick Sort Result: [18, 22, 42, 46, 66, 70, 79, 82, 83, 84]

Reverse Sorted List: [84, 83, 82, 79, 70, 66, 46, 42, 22, 18]
Merge Sort Result: [18, 22, 42, 46, 66, 70, 79, 82, 83, 84]
Quick Sort Result: [18, 22, 42, 46, 66, 70, 79, 82, 83, 84]
```

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

• Use AI to suggest the most efficient search and sort algorithms for this use case.

• Implement the recommended algorithms in Python.

• Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

• A table mapping operation → recommended algorithm → justification.

• Working Python functions for searching and sorting the inventory.

Prompt :

A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity.Store staff need to:1. Quickly search for a product by ID or name.2. Sort products by price or quantity for stock analysis. analyse the above senario and implement the recommended algorithms in Python. and result should be in A table mapping operation → recommended algorithm → justification.

```python
class SimpleDataFrame:
    def __init__(self, data):
        # data is expected to be a dict of column -> list-of-values
        self.columns = list(data.keys())
        # transpose to rows; handle unequal column lengths by filling with empty strings
        cols_as_lists = [list(data[col]) for col in self.columns]
        max_len = max((len(col) for col in cols_as_lists), default=0)
        for col in cols_as_lists:
            col += [""] * (max_len - len(col))
        self.rows = list(zip(*cols_as_lists))

    def __str__(self):
        # compute column widths
        widths = [
            max(len(str(self.columns[i])), max((len(str(row[i])) for row in self.rows), default=0))
            for i in range(len(self.columns))
        ]
        header = " | ".join(self.columns[i].ljust(widths[i]) for i in range(len(self.columns)))
        sep = "-+-".join("-" * widths[i] for i in range(len(self.columns)))
        lines = [header, sep]
        for row in self.rows:
            lines.append(" | ".join(str(row[i]).ljust(widths[i]) for i in range(len(row))))
        return "\n".join(lines)

    __repr__ = __str__

# provide a pd-like namespace with DataFrame constructor used later in the cell
pd = type("pd", (), {"DataFrame": SimpleDataFrame})
class Product:
    def __init__(self, product_id, name, price, stock_quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock_quantity = stock_quantity
    def __repr__(self):
        return f"{self.name} (ID: {self.product_id}, Price: {self.price}, Stock: {self.stock_quantity})"
class Inventory:
    def __init__(self):
        self.products = []
    def add_product(self, product):
        self.products.append(product)
```

✓ 0.0s

Output :

```
✓ 0.0s

Search by ID (1): Laptop (ID: 1, Price: 999.99, Stock: 10)
Search by Name (Smartphone): Smartphone (ID: 2, Price: 499.99, Stock: 50)

Products sorted by Price:
Headphones (ID: 3, Price: 199.99, Stock: 100)
Smartphone (ID: 2, Price: 499.99, Stock: 50)
Laptop (ID: 1, Price: 999.99, Stock: 10)

Products sorted by Stock Quantity:
Laptop (ID: 1, Price: 999.99, Stock: 10)
Smartphone (ID: 2, Price: 499.99, Stock: 50)
Headphones (ID: 3, Price: 199.99, Stock: 100)

Operation → Recommended Algorithm → Justification:
Operation              | Recommended Algorithm   | Justification
-----------------------+-------------------------+-------------------------------------------------------------------------------------
Search by ID           | Hash Table (Dictionary) | Provides O(1) average time complexity for search operations.
Search by Name         | Hash Table (Dictionary) | Provides O(1) average time complexity for search operations.
Sort by Price          | Merge Sort              | Merge Sort is efficient for sorting large datasets with O(n log n) time complexity.
Sort by Stock Quantity | Merge Sort              | Merge Sort is efficient for sorting large datasets with O(n log n) time complexity.
```

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

• Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).

• Implement sorting algorithms to rank stocks by percentage change.

• Implement a search function that retrieves stock data instantly when a stock symbol is entered.

• Optimize sorting with Heap Sort and searching with Hash Maps.

• Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

Prompt :

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.analyse the above senario and implement stock price data (Stock Symbol, Opening Price, Closing Price) and sorting algorithms to rank stocks by percentage change.Optimize sorting with Heap Sort and searching with Hash Maps.

```python
#An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily ga
#analyse the above senario and implement stock price data (Stock Symbol, Opening Price, Closing Price) and sorting algorithms to rank stocks by percentage
class Stock:
    def __init__(self, symbol, opening_price, closing_price):
        self.symbol = symbol
        self.opening_price = opening_price
        self.closing_price = closing_price
    def percentage_change(self):
        return ((self.closing_price - self.opening_price) / self.opening_price) * 100
    def __repr__(self):
        return f"{self.symbol} (Opening: {self.opening_price}, Closing: {self.closing_price}, Change: {self.percentage_change():.2f}%)"
class StockAnalyzer:
    def __init__(self):
        self.stocks = []
    def add_stock(self, stock):
        self.stocks.append(stock)
    def search_by_symbol(self, symbol):
        # Using a dictionary for O(1) average time complexity search
        stock_dict = {stock.symbol: stock for stock in self.stocks}
        return stock_dict.get(symbol, "Stock not found")
    def heap_sort_by_percentage_change(self):
        # Using Heap Sort to sort stocks by percentage change
        import heapq
        return heapq.nlargest(len(self.stocks), self.stocks, key=lambda x: x.percentage_change())
# Sample stock data
analyzer = StockAnalyzer()
analyzer.add_stock(Stock("AAPL", 150.00, 155.00))
analyzer.add_stock(Stock("GOOGL", 2800.00, 2750.00))
analyzer.add_stock(Stock("AMZN", 3400.00, 3450.00))
# Search for a specific stock symbol
print("Search for AAPL:", analyzer.search_by_symbol("AAPL"))
print("Search for MSFT:", analyzer.search_by_symbol("MSFT"))
# Sort stocks by percentage change
print("\nStocks sorted by Percentage Change:")
for stock in analyzer.heap_sort_by_percentage_change():
    print(stock)
```

Output :

```
✓ 0.0s

Search for AAPL: AAPL (Opening: 150.0, Closing: 155.0, Change: 3.33%)
Search for MSFT: Stock not found

Stocks sorted by Percentage Change:
AAPL (Opening: 150.0, Closing: 155.0, Change: 3.33%)
AMZN (Opening: 3400.0, Closing: 3450.0, Change: 1.47%)
GOOGL (Opening: 2800.0, Closing: 2750.0, Change: -1.79%)

Operation → Recommended Algorithm → Justification:
Operation                 | Recommended Algorithm | Justification
-------------------------+----------------------+-----------------------------------------------------------------------------------------------------------------
Search by Stock Symbol    | Hash Map (Dictionary) | Provides O(1) average time complexity for searching stock symbols.
Sort by Percentage Change | Heap Sort             | Heap Sort efficiently retrieves the top stocks based on percentage change with O(n log k) time complexity.
```